Manual SQL Injection Testing Using DVWA

Objective

Perform manual SQL injection testing on DVWA to identify vulnerable input, retrieve database information, and examine Boolean-based and time-based blind SQL injection.

Initial Configuration

Step 1: Start the DVWA environment

Start the Windows VM hosting DVWA.

Open XAMPP Control Panel and start **Apache** and **MySQL**.

Open DVWA in the browser:

```
http://localhost/DVWA/
```

When accessing it from Kali, replace `localhost` with the Windows VM's IP address.

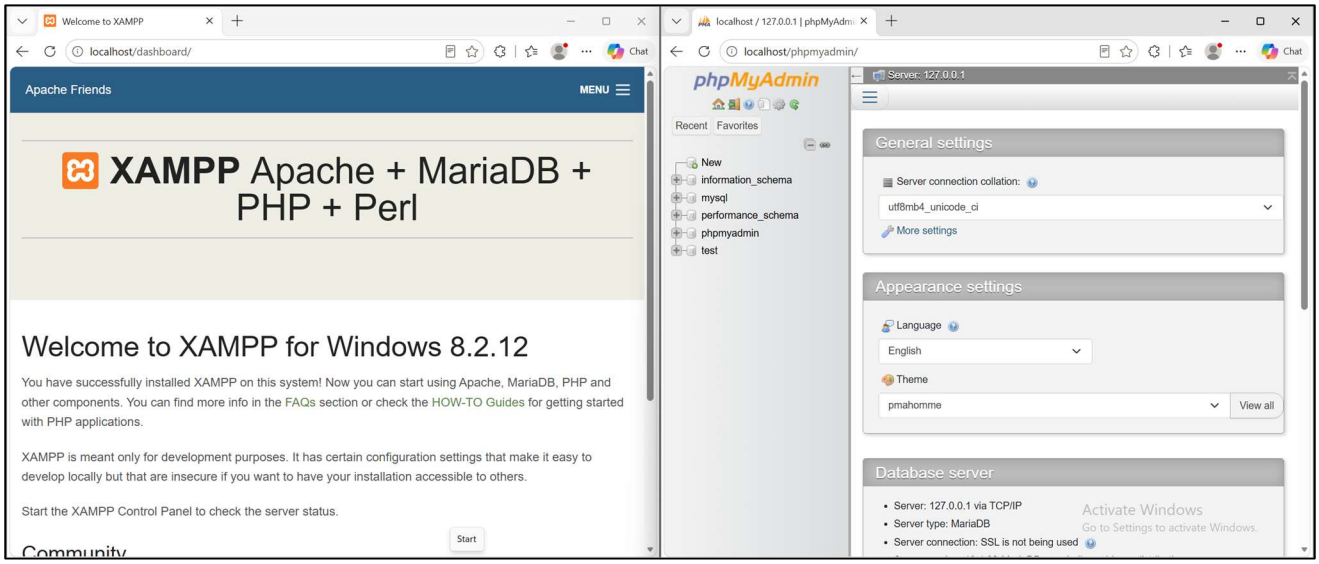


Figure 13: DVWA accessible with Apache and MySQL running.

Step 2: Log in to DVWA

Enter the DVWA login credentials. For a default installation:

Field	Value
Username	admin
Password	password

Click **Login**.

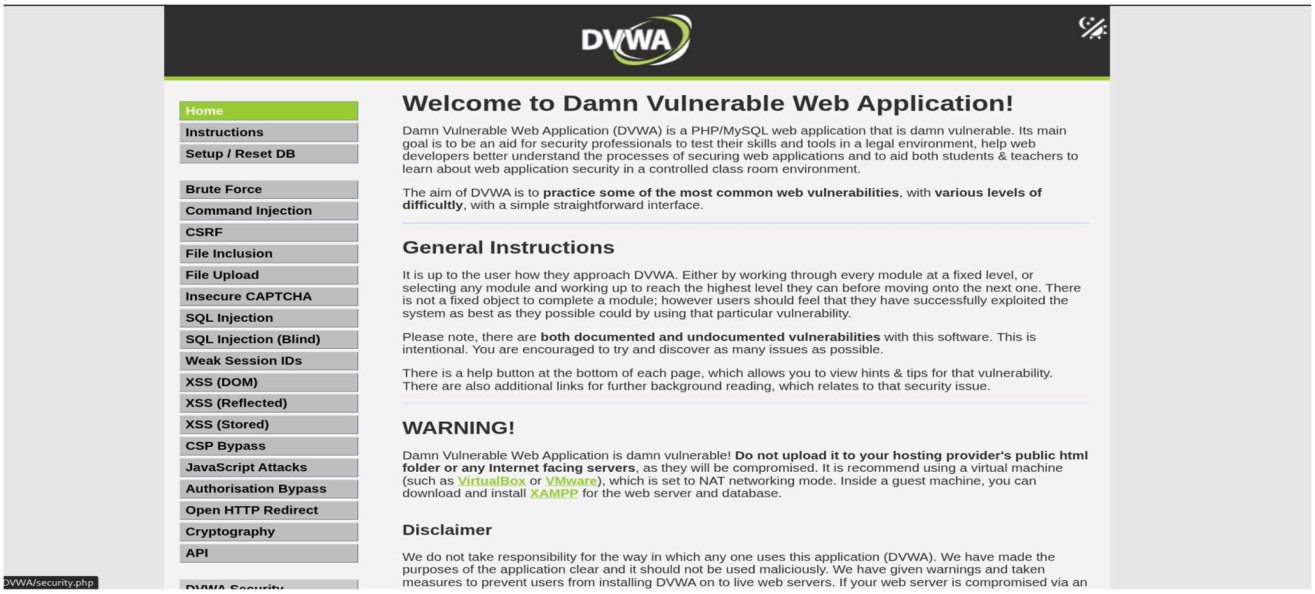


Figure 14: Successful login to DVWA.

Step 3: Set the security level

Open **DVWA Security**.

Select **Low** and click **Submit**.

Confirm that the current security level is Low.



Figure 15: DVWA security level set to Low.

Procedure

Part A: Basic and UNION-Based SQL Injection

Step 1: Submit a normal User ID

Open **SQL Injection** from the navigation menu.

Enter:

```
1
```

Click **Submit** and examine the returned first name and surname.

Use this response as the baseline for comparison.



Figure 16: Normal application response for User ID 1.

Step 2: Check quotation-mark handling

Enter:

```
'
```

Click **Submit**.

Check whether the application displays a database error. An error suggests that the input affects query syntax, but further testing is needed to confirm SQL injection.

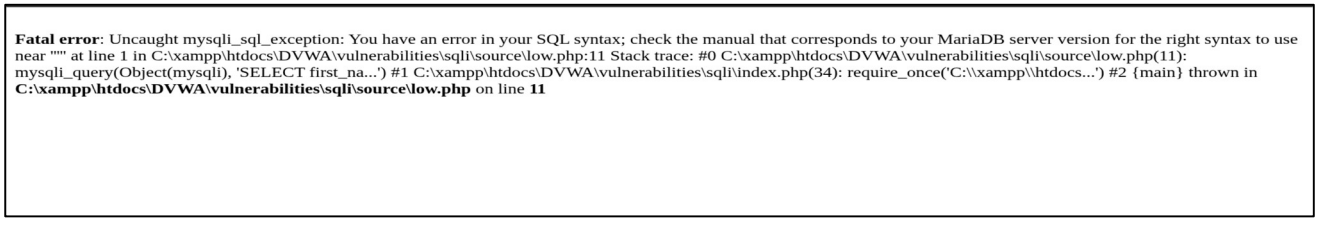


Figure 17: Application response to a single quotation mark.

Step 3: Submit a true condition

Enter:

```
1' AND '1'='1' -- -
```

Click **Submit**.

Check whether the record for User ID 1 appears because the additional condition is true.

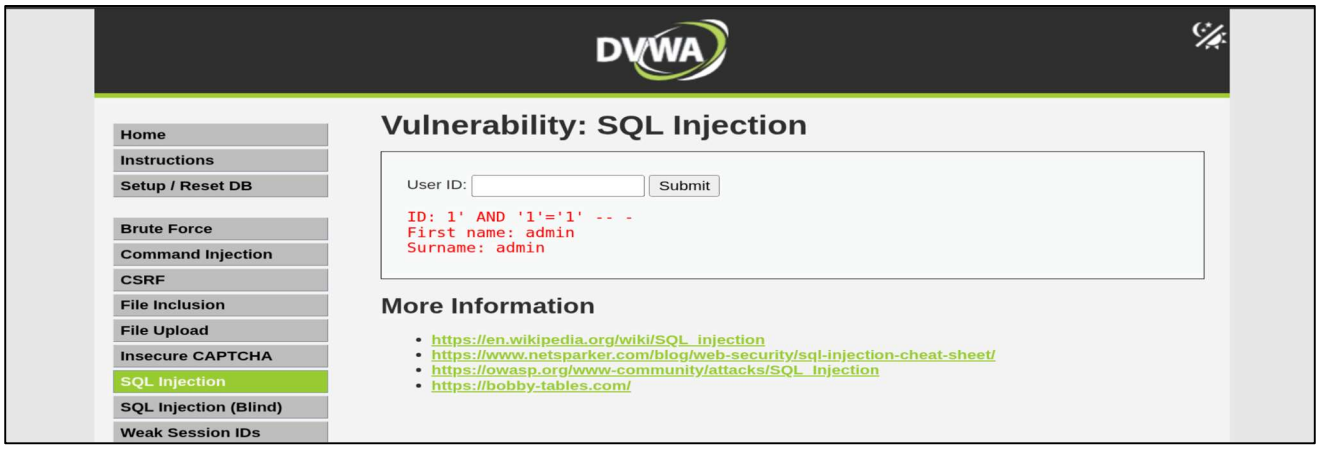


Figure 18: Response to an injected true condition.

Step 4: Submit a false condition

Enter:

```
1' AND '1'='2' -- -
```

Click **Submit**.

Compare the output with the previous response. No record is expected because the additional condition is false.

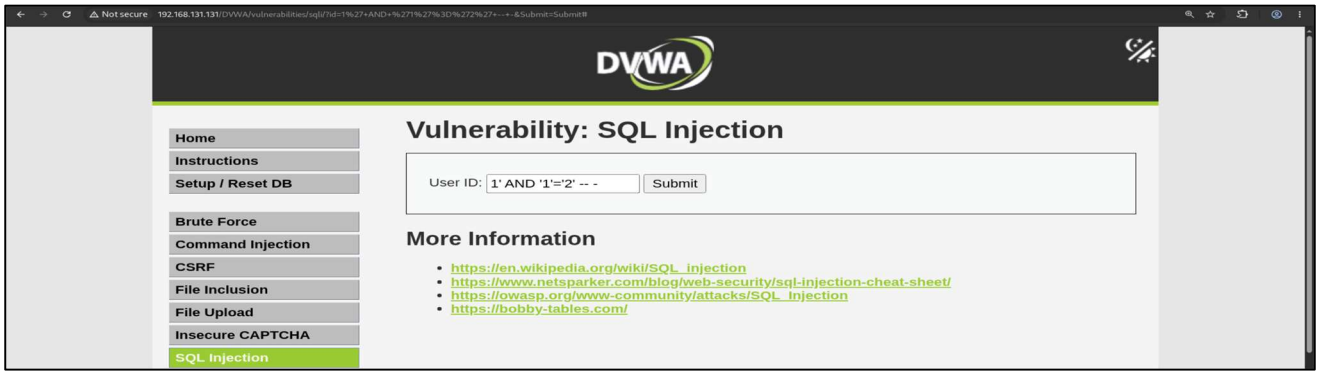


Figure 19: onse to an injected false condition.

Step 5: Retrieve multiple user records

Enter:

```
1' OR '1'='1' -- -
```

Click **Submit**.

Examine whether multiple user records are returned. The `OR` condition makes the query condition true for every row.

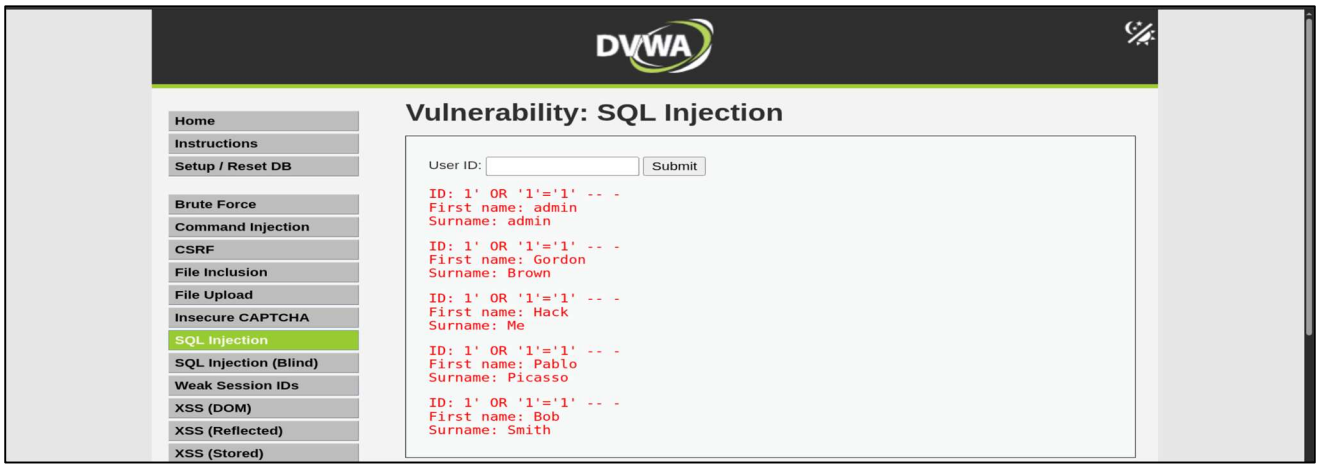


Figure 20: Multiple user records returned using an always-true condition.

Step 6: Test the first column position

Enter:

```
1' ORDER BY 1 -- -
```

Click **Submit**.

Check that the query executes without a column-position error.

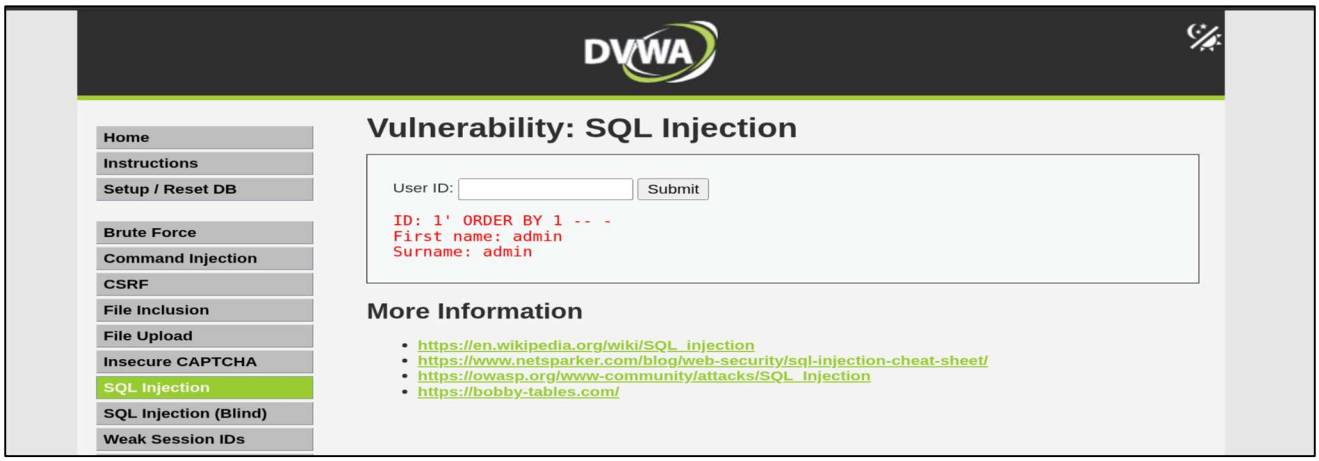


Figure 21: Successful query using ORDER BY 1.

Step 7: Test the second column position

Enter:

```
1' ORDER BY 2 -- -
```

Click **Submit**.

Check that the query executes without a column-position error.

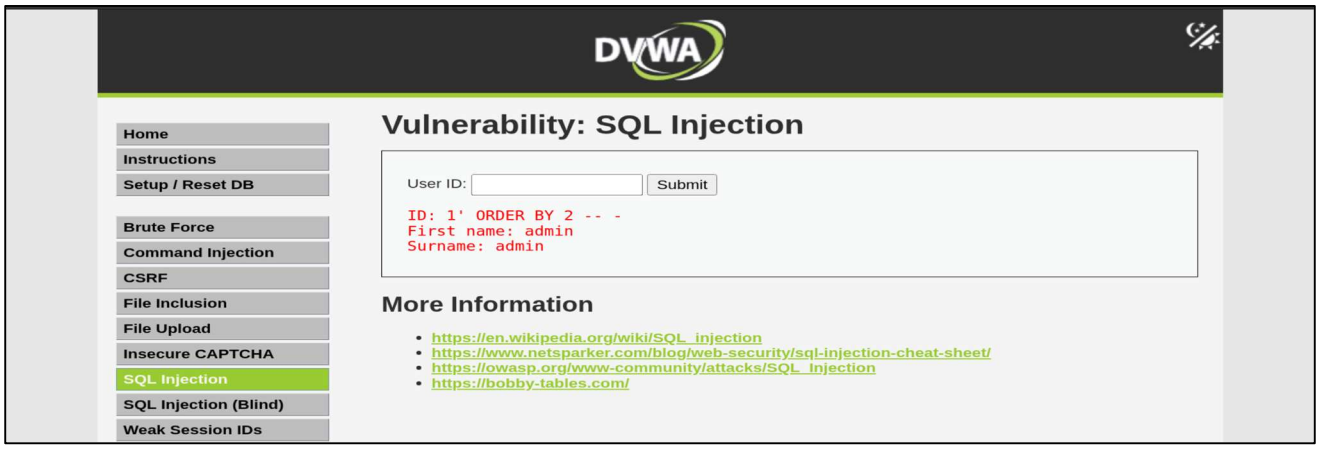


Figure 22: Successful query using ORDER BY 2.

Step 8: Test the third column position

Enter:

```
1' ORDER BY 3 -- -
```

Click **Submit**.

In the standard DVWA Low SQL Injection page, this produces a column-position error. Successful execution with `ORDER BY 2` and failure with `ORDER BY 3` indicate two output columns.

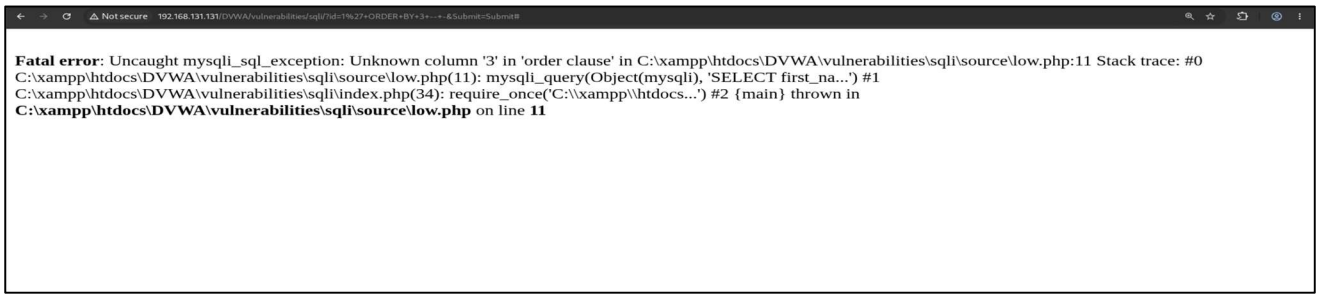


Figure 23: Error produced when referencing a third output column.

Step 9: Retrieve the database name and version

Enter:

```
-1' UNION SELECT database(), version() -- -
```

Click **Submit**.

Record the database name and database server version displayed in the result fields.

The value `-1` avoids matching an ordinary DVWA user record.

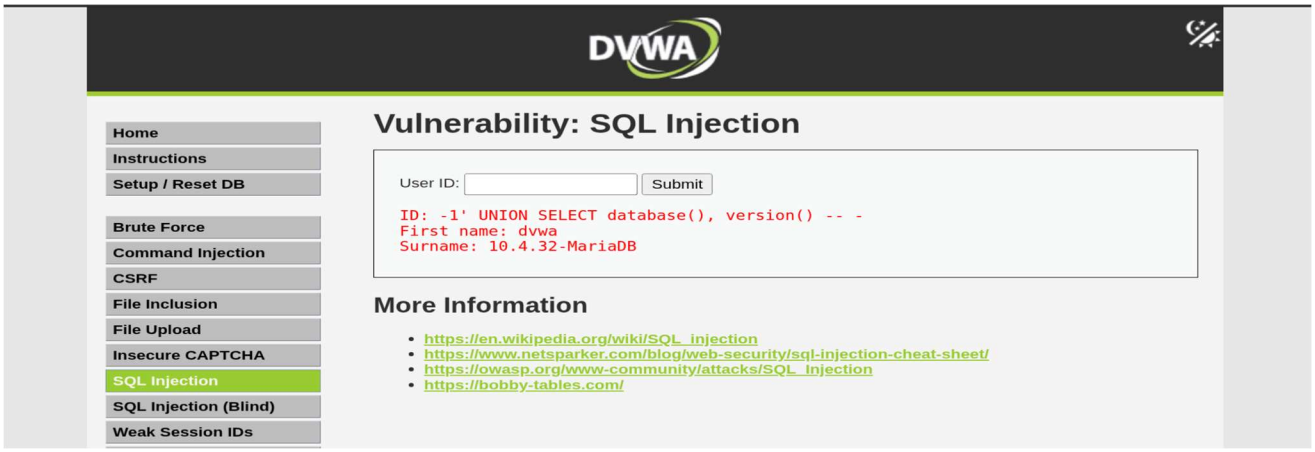


Figure 24: Database name and server version retrieved through UNION SELECT.

Step 10: Retrieve table names

Enter:

```
-1' UNION SELECT table_name, NULL FROM information_schema.tables
WHERE table_schema=database() -- -
```

Click **Submit**.

Examine the table names returned from the current database.

The `NULL` value provides the second column required to match the original query.

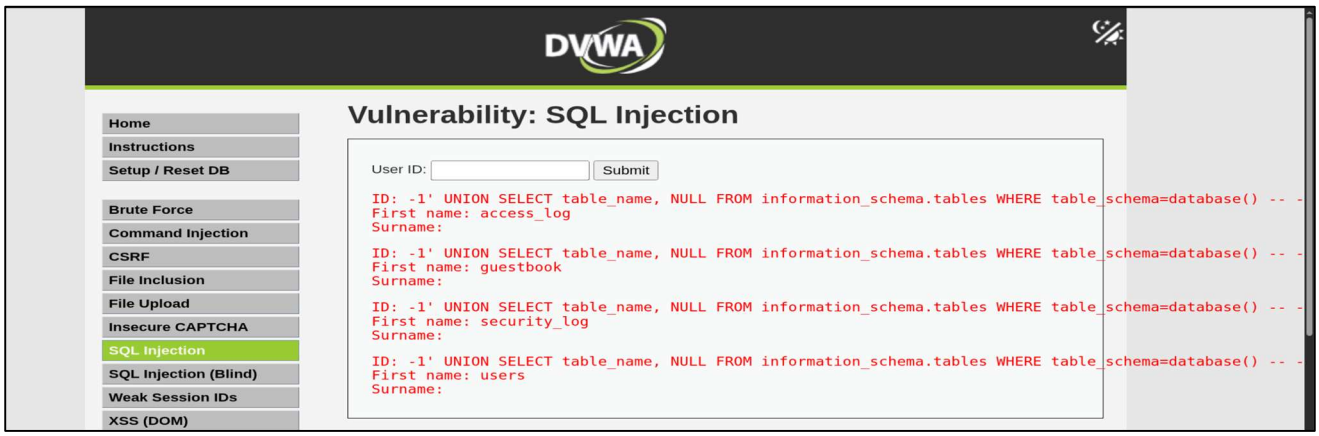


Figure 25: Table names retrieved from the current database.

Step 11: Retrieve column names

Enter:

```
-1' UNION SELECT column_name, data_type FROM
information_schema.columns WHERE table_schema=database() AND
table_name='users' -- -
```

Click **Submit**.

Record the column names and data types belonging to the `users` table.

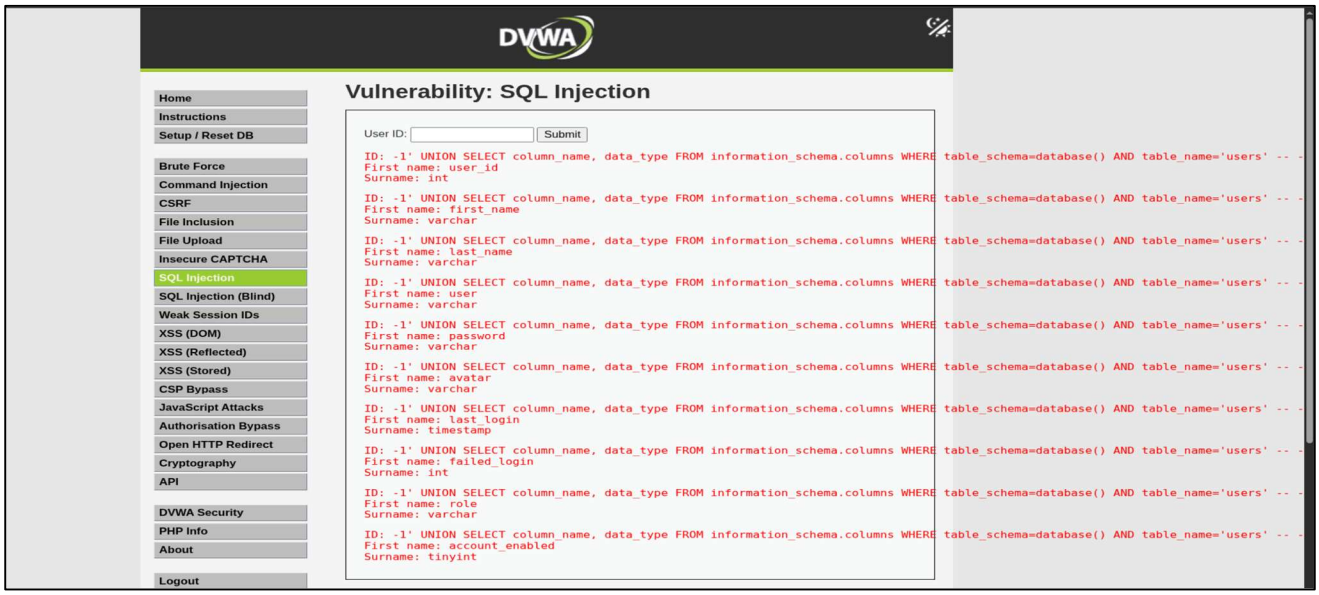


Figure 26: Column names and data types of the users table.

Step 12: Retrieve sample account information

Enter:

```
-1' UNION SELECT user, password FROM users -- -
```

Click **Submit**.

Examine the sample usernames and stored password hashes returned by DVWA.

The displayed hashes are not plaintext passwords.

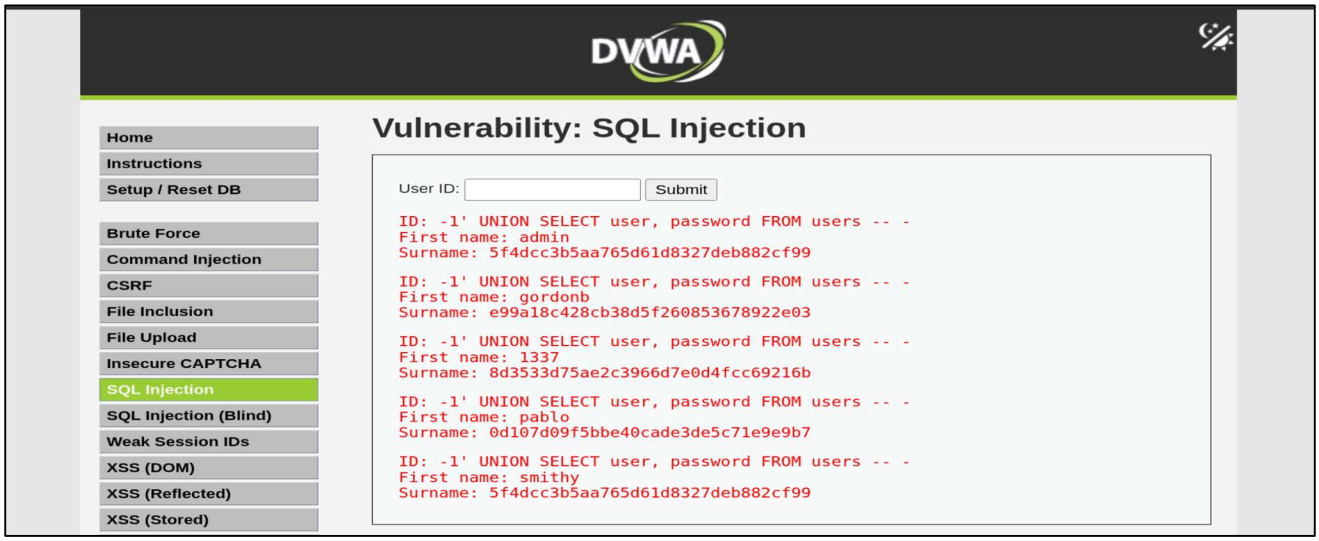


Figure 27: DVWA sample usernames and password hashes retrieved.

Part B: Boolean-Based Blind SQL Injection

Step 13: Record the normal blind SQL injection response

Open **SQL Injection (Blind)**.

Enter:

```
1
```

Click **Submit**.

Check that the application indicates that the user exists without displaying the user's details.

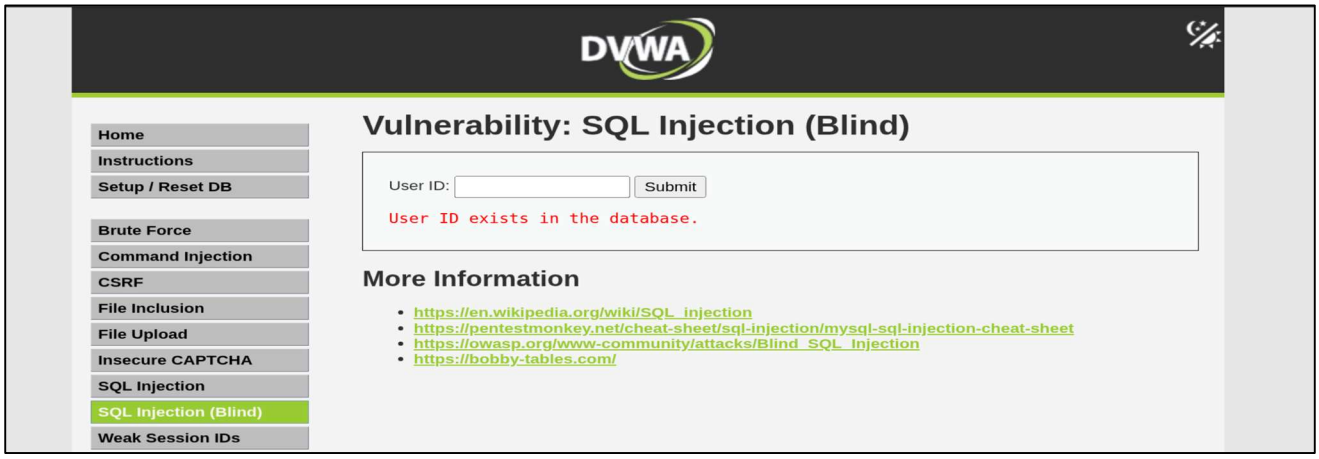


Figure 28: Normal response for an existing User ID on the blind SQL injection page.

Step 14: Test a true Boolean condition

Enter:

```
1' AND 1=1 -- -
```

Click **Submit**.

Record the response. The application should indicate that the user exists.

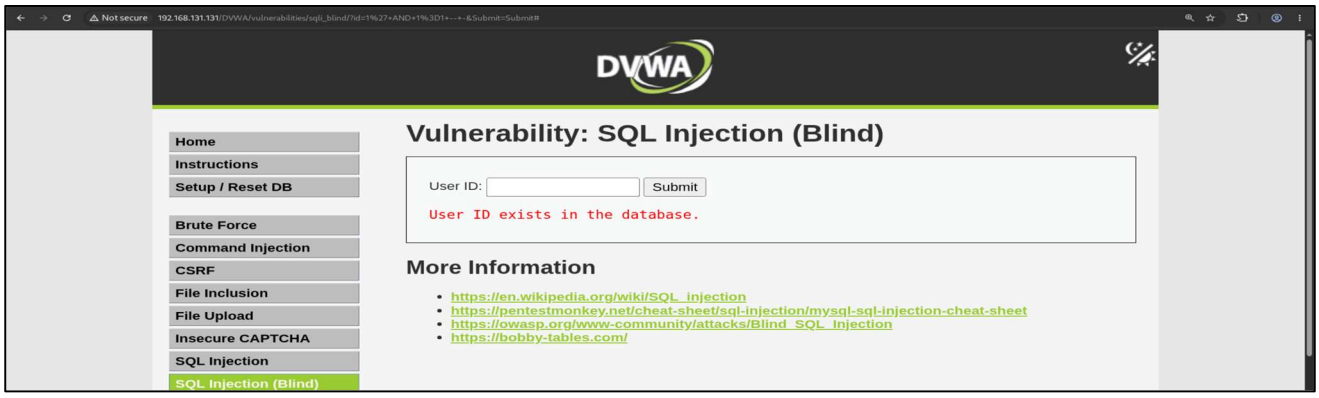


Figure 29: Blind SQL injection response for a true condition.

Step 15: Test a false Boolean condition

Enter:

```
1' AND 1=2 -- -
```

Click **Submit**.

Record the response. The application should indicate that the user is missing.

Compare this with the previous response to identify the true/false response difference.

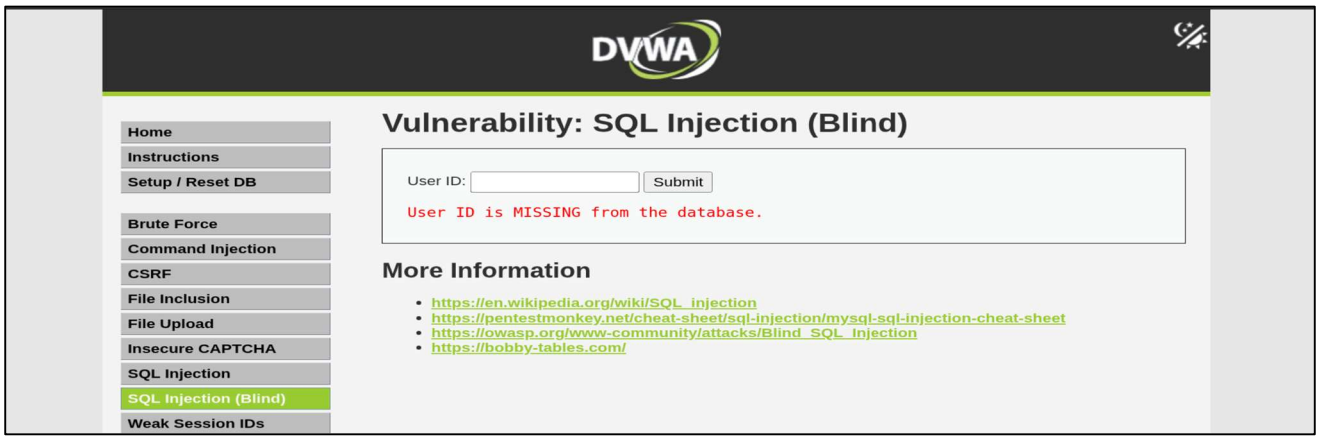


Figure 30: Blind SQL injection response for a false condition.

Step 16: Infer a database-name character

Enter:

```
1' AND SUBSTRING(database(),1,1)='d' -- -
```

Click **Submit**.

If the database name begins with `d`, the application should indicate that the user exists. Otherwise, it should indicate that the user is missing.

This demonstrates how database information can be inferred without directly displaying query results.

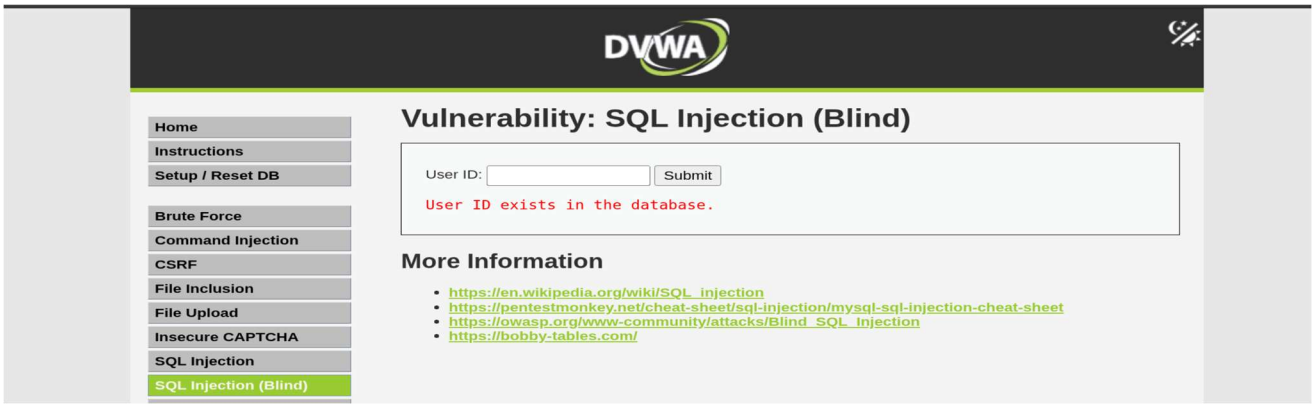


Figure 31: First character of the database name checked using a Boolean condition.

Part C: Time-Based Blind SQL Injection

Step 17: Prepare to measure response time

Remain on the **SQL Injection (Blind)** page.

Press **F12** to open browser Developer Tools and select **Network**. Enable **Preserve log** if needed to retain requests during page navigation.

Identify the request generated when the form is submitted and inspect its duration.

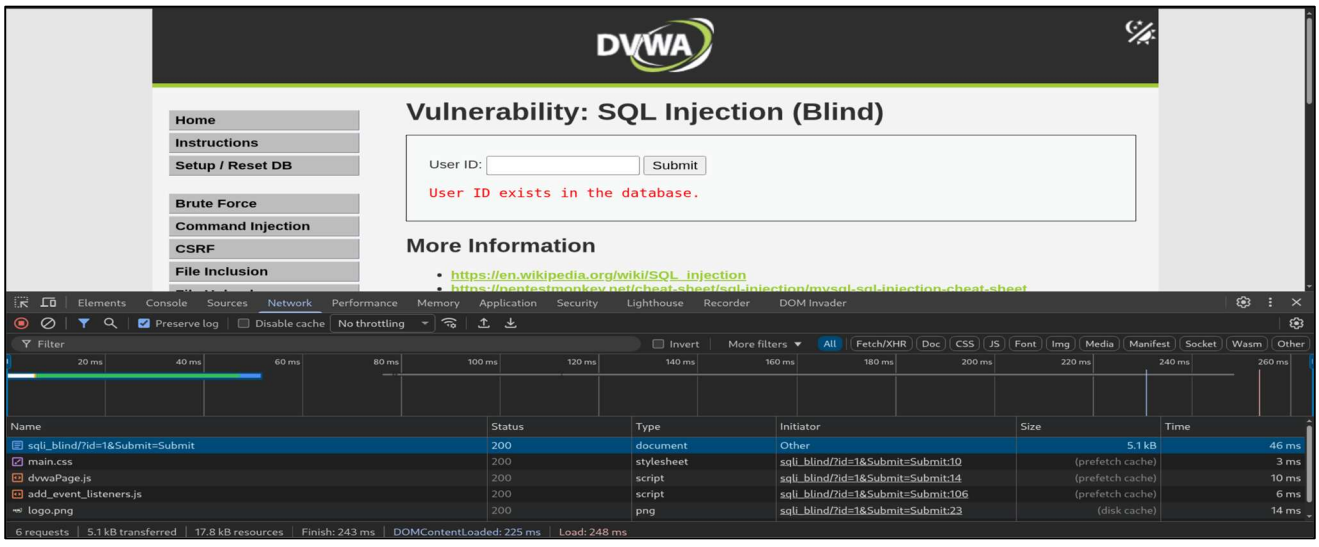


Figure 32: Browser Network tab prepared to record request duration.

Step 18: Submit a query with an intentional delay

Enter:

```
1' AND IF(1=1,SLEEP(3),0) -- -
```

Click **Submit**.

Inspect the request duration in the Network tab. The true condition should introduce approximately three seconds of additional delay.

Record the measured duration.

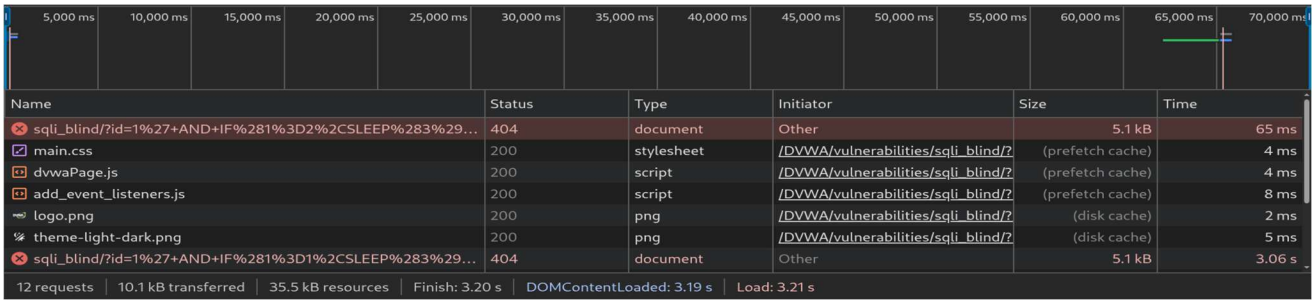


Figure 33: Figure 33: Request duration for the condition that executes SLEEP(3).

Step 19: Submit a query without an intentional delay

Enter:

```
1' AND IF(1=2,SLEEP(3),0) -- -
```

Click **Submit**.

Inspect the request duration. The false condition should not execute the delay.

Repeat both tests to check that the timing difference is consistent. The page message may be identical; the timing difference is the evidence.

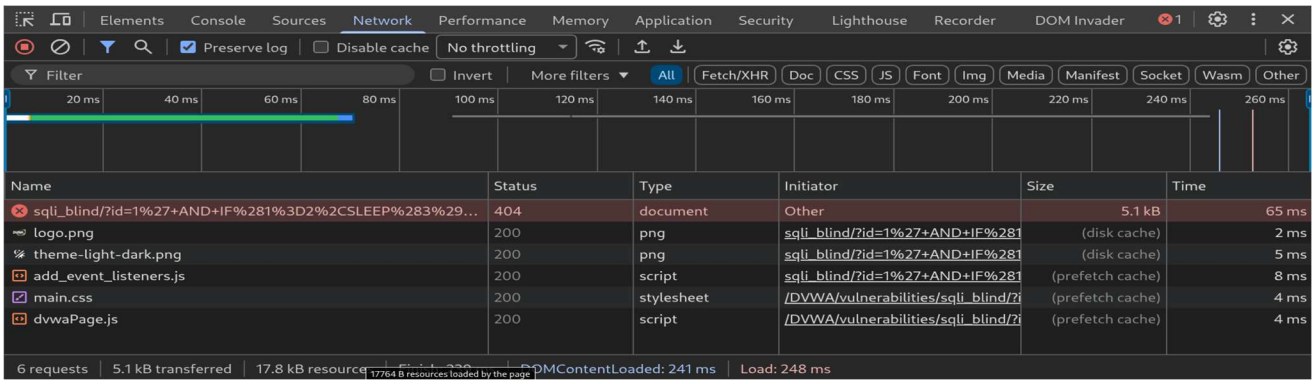


Figure 34: Request duration for the condition that does not execute SLEEP(3).

Result

The exercise demonstrates manual testing for basic, UNION-based, Boolean-based blind and time-based blind SQL injection in DVWA. Successful techniques and retrieved information should be documented using the observed responses and corresponding screenshots.